

SKILL WEEK 6

1. To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency.

Create a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill6.1.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |
```

Write the code:

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_HISTORY 5
#define MAX_COMMAND 100

char history[MAX_HISTORY][MAX_COMMAND];
int count = 0;
int next = 0;

void add_command(const char *command)
{
    strncpy(history[next], command, MAX_COMMAND - 1);
    history[next][MAX_COMMAND - 1] = '\0';

    next = (next + 1) % MAX_HISTORY;

    if (count < MAX_HISTORY)
        count++;
}

void display_history()
{
    if (count == 0)
    {
        printf("History is empty.\n");
        return;
    }

    int start = (count == MAX_HISTORY) ? next : 0;

    for (int i = 0; i < count; i++)
    {
        int index = (start + i) % MAX_HISTORY;
        printf("%d %s\n", i + 1, history[index]);
    }
}

void retrieve_command(int position)
{
    if (position < 1 || position > count)
    {
        printf("Invalid history entry.\n");
        return;
    }

    int start = (count == MAX_HISTORY) ? next : 0;
    int index = (start + position - 1) % MAX_HISTORY;
```

```

    printf("Command %d: %s\n", position, history[index]);
}

int validate_history()
{
    if (count < 0 || count > MAX_HISTORY)
        return 0;

    if (next < 0 || next >= MAX_HISTORY)
        return 0;

    for (int i = 0; i < count; i++)
    {
        int start = (count == MAX_HISTORY) ? next : 0;
        int index = (start + i) % MAX_HISTORY;

        if (history[index][0] == '\0')
            return 0;
    }

    return 1;
}

int main()
{
    char command[MAX_COMMAND];
    char input[MAX_COMMAND];

    printf("Command History Manager\n");

    while (1)
    {
        printf("history-shell$ ");
        fflush(stdout);

        if (fgets(command, sizeof(command), stdin) == NULL)
            break;

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
            break;

        if (strcmp(command, "history") == 0)
        {
            display_history();
            continue;
        }
    }
}

```

```

    if (strncmp(command, "get ", 4) == 0)
    {
        int position = atoi(command + 4);
        retrieve_command(position);
        continue;
    }

    if (strcmp(command, "check") == 0)
    {
        if (validate_history())
            printf("History consistency: VALID\n");
        else
            printf("History consistency: INVALID\n");

        continue;
    }

    if (strlen(command) == 0)
        continue;

    add_command(command);
    printf("Command stored.\n");
}

return 0;
}

```

OUTPUT:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill6.1.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc skill6.1.c -o skill6.1
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill6.1
Command History Manager
history-shell$ pwd
Command stored.
history-shell$ ls
Command stored.
history-shell$ history
1  pwd
2  ls
history-shell$ exit
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |

```

2. To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations

Create a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill6.2.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |
```

Write the code:

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_COMMANDS 5
#define MAX_ARGS 10
#define MAX_INPUT 512

typedef struct
{
    char *args[MAX_ARGS];
    int argc;
} Command;

typedef struct
{
    Command commands[MAX_COMMANDS];
    int count;
} Pipeline;

void parse_command(char *text, Command *command)
{
    command->argc = 0;

    char *token = strtok(text, " ");

    while (token != NULL && command->argc < MAX_ARGS - 1)
    {
        command->args[command->argc++] = token;
        token = strtok(NULL, " ");
    }

    command->args[command->argc] = NULL;
}

int validate_pipeline(Pipeline *pipeline)
{
    if (pipeline->count <= 0 || pipeline->count > MAX_COMMANDS)
        return 0;

    for (int i = 0; i < pipeline->count; i++)
    {
        if (pipeline->commands[i].argc == 0)
            return 0;
    }

    return 1;
}
```

```

}

void display_pipeline(Pipeline *pipeline)
{
    printf("\nPipeline layout:\n");

    for (int i = 0; i < pipeline->count; i++)
    {
        printf("Stage %d: ", i + 1);

        for (int j = 0; j < pipeline->commands[i].argc; j++)
        {
            printf("%s", pipeline->commands[i].args[j]);

            if (j < pipeline->commands[i].argc - 1)
                printf(" ");
        }

        printf("\n");

        if (i < pipeline->count - 1)
            printf("      |\n      v\n");
    }
}

void execute_pipeline(Pipeline *pipeline)
{
    int pipes[MAX_COMMANDS - 1][2];
    pid_t pids[MAX_COMMANDS];

    for (int i = 0; i < pipeline->count - 1; i++)
    {
        if (pipe(pipes[i]) == -1)
        {
            perror("pipe");
            return;
        }
    }

    for (int i = 0; i < pipeline->count; i++)
    {
        pids[i] = fork();

        if (pids[i] == -1)
        {
            perror("fork");
            return;
        }
    }
}

```

```

        if (pids[i] == 0)
        {
            if (i > 0)
            {
                if (dup2(pipes[i - 1][0], STDIN_FILENO) == -1)
                {
                    perror("dup2");
                    exit(1);
                }
            }

            if (i < pipeline->count - 1)
            {
                if (dup2(pipes[i][1], STDOUT_FILENO) == -1)
                {
                    perror("dup2");
                    exit(1);
                }
            }

            for (int j = 0; j < pipeline->count - 1; j++)
            {
                close(pipes[j][0]);
                close(pipes[j][1]);
            }

            execvp(pipeline->commands[i].args[0],
                    pipeline->commands[i].args);

            perror("execvp");
            exit(1);
        }
    }

    for (int i = 0; i < pipeline->count - 1; i++)
    {
        close(pipes[i][0]);
        close(pipes[i][1]);
    }

    for (int i = 0; i < pipeline->count; i++)
        waitpid(pids[i], NULL, 0);
}

int main()
{
    char input[MAX_INPUT];

    printf("Pipeline Manager\n");

```

```

printf("Enter commands separated by |.\n");
printf("Example: ls -l | grep .c | sort\n");
printf("Type exit to quit.\n\n");

while (1)
{
    Pipeline pipeline;
    pipeline.count = 0;

    printf("pipeline-shell$ ");
    fflush(stdout);

    if (fgets(input, sizeof(input), stdin) == NULL)
        break;

    input[strcspn(input, "\n")] = '\0';

    if (strcmp(input, "exit") == 0)
        break;

    if (strlen(input) == 0)
        continue;

    char *part = strtok(input, "|");

    while (part != NULL && pipeline.count < MAX_COMMANDS)
    {
        while (*part == ' ')
            part++;

        parse_command(part, &pipeline.commands[pipeline.count]);
        pipeline.count++;

        part = strtok(NULL, "|");
    }

    if (!validate_pipeline(&pipeline))
    {
        printf("Invalid pipeline configuration.\n");
        continue;
    }

    display_pipeline(&pipeline);

    printf("\nExecuting pipeline...\n");

    execute_pipeline(&pipeline);

    printf("\nPipeline completed.\n");

```

```

    }

    return 0;
}

```

OUTPUT:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill6.2.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc skill6.2.c -o skill6.2
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill6.2
Pipeline Manager
Enter commands separated by |.
Example: ls -l | grep .c | sort
Type exit to quit.

pipeline-shell$ uname

Pipeline layout:
Stage 1: uname

Executing pipeline...
Linux

Pipeline completed.
pipeline-shell$ pwd

Pipeline layout:
Stage 1: pwd

Executing pipeline...
/home/vishwa_teja_veerandi

Pipeline completed.
pipeline-shell$ exit
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |
```